Ngày 1: Kiểu dữ liệu, value và reference

Oct 15, 2026 · @Hoàng Tử Hói

Kế hoạch buổi học (2,5 giờ)

Mục tiêu hôm nay: nhìn một dòng gán hoặc so sánh bất kỳ và biết ngay nó copy giá trị hay copy "địa chỉ". Đây là gốc của rất nhiều lỗi React kiểu "setState rồi mà UI không cập nhật" hay "useEffect chạy vô hạn".

Thời gian	Việc
15 phút	Chuẩn bị môi trường (bên dưới)
40 phút	Đọc bài giảng, gõ lại từng ví dụ vào file, không copy-paste
75 phút	Làm 3 phần bài tập
20 phút	Ghi nhật ký và nộp bài

Chuẩn bị môi trường:

1. Tắt GitHub Copilot hoặc mọi AI autocomplete trong VS Code. Trong tháng 1, em chỉ hỏi AI để giải thích khái niệm.
2. Kiểm tra `node -v` (cần Node 20 trở lên).
3. Tạo repo GitHub tên `learning-log`, trong đó tạo thư mục `js-core/day-01/`.
4. Chạy mỗi file bài tập bằng `node ten-file.js`.

Bài giảng

1. Hai nhóm kiểu dữ liệu

JavaScript có 7 kiểu primitive: `string`, `number`, `bigint`, `boolean`, `undefined`, `null`, `symbol`. Mọi thứ còn lại là object: object thường, array, function, Date, Map, Set.

Hãy hình dung biến là một cái hộp. Với primitive, trong hộp chứa chính giá trị. Với object, trong hộp chỉ chứa tờ giấy ghi địa chỉ căn nhà (reference); căn nhà thật nằm chỗ khác trong bộ nhớ.

```
let a = 10;
let b = a;      // copy giá trị 10
b = 20;
console.log(a); // 10

const user1 = { name: "Hùng" };
const user2 = user1;  // copy ĐỊA CHỈ, cả hai cùng trỏ một căn nhà
user2.name = "Lan";
console.log(user1.name); // "Lan"
```

Primitive là bất biến: `str.toUpperCase()` không sửa `str` mà trả về chuỗi mới. Object thì sửa được tại chỗ (mutate).

2. const không có nghĩa là không đổi

`const` chỉ khóa cái hộp, không khóa căn nhà. Em không gán lại được biến, nhưng vẫn sửa được nội dung object.

```
const list = [1, 2];
list.push(3);   // OK, list là [1, 2, 3]
list = [];      // TypeError: Assignment to constant variable
```

3. typeof và những cái bẫy

```
typeof "hi"         // "string"
typeof 42           // "number"
typeof undefined    // "undefined"
typeof null         // "object"   <- lỗi lịch sử của JS, nhớ thuộc
typeof []           // "object"   <- kiểm tra mảng bằng Array.isArray([])
typeof function(){} // "function"
```

4. So sánh: === so địa chỉ, không so nội dung

```
"abc" === "abc"       // true, so giá trị
[] === []             // false, hai căn nhà khác nhau
{ a: 1 } === { a: 1 } // false
const x = {}; const y = x;
x === y               // true, cùng một địa chỉ
NaN === NaN           // false; dùng Number.isNaN() hoặc Object.is(NaN, NaN)
```

5. Truyền vào hàm

JS luôn truyền bằng giá trị. Với object, giá trị được copy chính là địa chỉ. Vì vậy hàm sửa thuộc tính thì bên ngoài bị ảnh hưởng, nhưng gán lại tham số thì không.

```
function rename(u) { u.name = "Minh"; }
function replace(u) { u = { name: "Tú" }; }
const p = { name: "An" };
rename(p);  console.log(p.name); // "Minh"
replace(p); console.log(p.name); // vẫn "Minh"
```

6. Copy nông và copy sâu

```
const original = { name: "An", address: { city: "HN" } };

const shallow = { ...original };   // copy nông: chỉ tầng 1 là mới
shallow.address.city = "HCM";
console.log(original.address.city); // "HCM"  <- tầng 2 vẫn dùng chung

const deep = structuredClone(original); // copy sâu, không copy được function
```

7. Vì sao điều này quan trọng với React

React so sánh state cũ và mới bằng `Object.is`, tức là so địa chỉ. Nếu em mutate rồi set lại cùng một object, React thấy địa chỉ không đổi và bỏ qua render.

```
// SAI: mutate, cùng địa chỉ, UI không cập nhật
todos.push(newTodo);
setTodos(todos);

// ĐÚNG: tạo mảng mới
setTodos([...todos, newTodo]);
```

Tương tự, một object tạo mới trong mỗi lần render (ví dụ `const options = { page: 1 }`) có địa chỉ mới mỗi lần, nên `useEffect(..., [options])` sẽ chạy lại sau mọi lần render.

Câu cần nhớ: primitive so và copy theo giá trị; object so và copy theo địa chỉ. Muốn React thấy thay đổi, phải tạo object mới.

Bài tập

Phần A: Đoán output (25 phút)

Quy tắc: viết câu trả lời và một câu giải thích vào file `predictions.md` TRƯỚC, sau đó mới chạy code để kiểm tra. Câu nào sai thì ghi lại vì sao mình sai. Câu sai là câu có giá trị nhất.

```
// A1
let s = "react";
s[0] = "R";
console.log(s);

// A2
const a = [1, 2, 3];
const b = a;
b.push(4);
console.log(a.length);

// A3
console.log([1, 2] === [1, 2], "1" == 1, null == undefined, null ===
undefined);

// A4
const o = { n: 1 };
function f(x) { x.n++; x = { n: 100 }; x.n++; }
f(o);
console.log(o.n);

// A5
const arr = [{ id: 1, done: false }];
const copy = [...arr];
copy[0].done = true;
console.log(arr[0].done, arr === copy);

// A6
console.log(typeof null, typeof NaN, typeof [], Array.isArray([]));

// A7
console.log(new Set([{ a: 1 }, { a: 1 }]).size, new Set([1, 1, "1"]).size);

// A8
const user = { name: "An", tags: ["js"] };
const clone = structuredClone(user);
clone.tags.push("ts");
console.log(user.tags.length, clone.tags === user.tags);
```

Phần B: Code tay (30 phút)

Tạo file `immutable.js`. Không hàm nào được sửa dữ liệu đầu vào. Sau mỗi hàm, viết `console.log` chứng minh input cũ không đổi.

1. `shallowEqual(obj1, obj2)`: trả về `true` nếu hai object có cùng số key và mỗi key có giá trị `===` nhau. (Đây chính là cách `React.memo` so props.)
2. `updateCity(user, city)`: trả về user mới với `user.address.city` đã đổi. Kiểm tra: `user.address` cũ phải giữ nguyên thành phố cũ.
3. `toggleTodo(todos, id)`: trả về mảng mới, todo có `id` đó đảo `done`, các todo khác giữ nguyên địa chỉ cũ.
4. `removeById(todos, id)`: trả về mảng mới không có phần tử đó.

Phần C: Sửa bug React (20 phút)

Không cần chạy React. Đọc code, viết giải thích và code đã sửa vào file `react-bugs.md`.

```
// C1: Bấm nút "+" mà số lượng trên màn hình không đổi. Vì sao? Sửa thế nào?
function Cart() {
  const [items, setItems] = useState([{ id: 1, name: "Áo", qty: 1 }]);

  function increase(id) {
    const item = items.find((i) => i.id === id);
    item.qty += 1;
    setItems(items);
  }
  // ...render danh sách items và nút "+"
}

// C2: Tab Network cho thấy request gọi liên tục không dừng. Vì sao? Đưa ra 2 cách sửa.
function TaskList() {
  const [tasks, setTasks] = useState([]);
  const filters = { status: "open" };

  useEffect(() => {
    fetchTasks(filters).then(setTasks);
  }, [filters]);
  // ...
}
```

Nộp bài và nhật ký

Nộp trong thread của lộ trình, bằng một trong hai cách: dán link thư mục `day-01` trên GitHub, hoặc dán trực tiếp nội dung 3 file. Thầy sẽ chấm, chỉ ra chỗ hiểu sai và giao bài ngày 2.

- [ ] `predictions.md`: 8 câu phần A, mỗi câu gồm dự đoán, kết quả thật, và giải thích nếu sai

- [ ] `immutable.js`: 4 hàm phần B có log kiểm tra
- [ ] `react-bugs.md`: giải thích và code sửa cho C1, C2
- [ ] `log.md`: nhật ký 3 dòng (hôm nay hiểu gì, còn mơ hồ gì, mai làm gì)
- [ ] Commit với message theo chuẩn: `docs(js-core): day 01 value vs reference`

Tiêu chí đạt: đúng ít nhất 6/8 câu phần A ngay lần đoán đầu, và giải thích được C1 bằng từ "địa chỉ" chứ không phải "React bị lỗi". Đúng ít hơn 6 câu cũng không sao, mai thầy sẽ cho thêm bài ôn trước khi sang closure.

Nhận xét bài nộp (16/10)

Kết quả: Phần A đúng 2/8 (cần 6/8), Phần B làm tốt 3/4 hàm, Phần C hiểu sai nguyên nhân cả hai bug. Chưa qua cổng, nên ngày 2 là ngày ôn. Điểm thấp ngày đầu là bình thường: nó cho thầy chính xác chỗ cần vá.

Điểm tốt: `updateCity`, `toggleTodo`, `removeById` đều đúng và gọn. Đặc biệt `toggleTodo` giữ nguyên địa chỉ các todo không đổi, đúng kiểu code React chuẩn. Em cũng ghi cả dự đoán lẫn đáp án thật, đúng cách học.

Lỗi gốc duy nhất: em nghĩ nhiều thao tác "tạo ra bản mới" trong khi thực tế chúng trả về cùng địa chỉ. 5 trong 6 câu sai đều từ đây.

Câu	Em trả lời	Đúng	Vì sao
A1	"React"	"react"	String là primitive bất biến. Gán `s[0]` bị bỏ qua âm thầm.
A3	`"1" == 1` là false	true	`==` ép kiểu trước khi so: "1" thành 1. Vì vậy luôn dùng `===`.
A4	3	2	`x.n++` sửa object gốc (n = 2). Sau đó `x = {n: 100}` chỉ đổi hộp `x` sang căn nhà khác; `o` không liên quan nữa.
A5	false false	true false	Spread chỉ tạo mảng mới; object bên trong vẫn dùng chung địa chỉ.
A6	`typeof NaN` là object	"number"	NaN là một giá trị của kiểu number ("số không hợp lệ").
A7	1	2	Set so bằng địa chỉ. Hai `{a: 1}` viết riêng là hai căn nhà.

Phần B, `shallowEqual` có 1 bug: `shallowEqual({ a: undefined }, { b: 1 })` trả về `true` (thầy đã chạy thử), vì `obj2.a` không tồn tại cũng ra `undefined`. Cần kiểm tra key có trong obj2 bằng `Object.hasOwn(obj2, key)`. Thêm nữa, `every` hợp hơn `reduce` vì nó dừng ngay khi gặp khác biệt. Bài cũng còn thiếu các `console.log` chứng minh input không đổi.

C1, chỗ hiểu sai: `find` không tạo object mới. Nó trả về địa chỉ của chính object năm trong `items`. Vì vậy `item.qty += 1` đã sửa thẳng vào state. Sau đó `setItems(items)` đưa lại cùng địa chỉ, `Object.is` ra true, React bỏ qua render. Clone mảng `[...items]` sẽ làm UI cập nhật, nhưng object bên trong vẫn bị mutate, gây lỗi với `memo` và StrictMode. Cách chuẩn:

```
setItems((prev) =>
  prev.map((i) => (i.id === id ? { ...i, qty: i.qty + 1 } : i))
);
```

C2, chỗ hiểu sai: effect không hề sửa `filters`. Nguyên nhân là vòng lặp: mỗi lần render tạo `filters` mới (địa chỉ mới), effect chạy, `setTasks` gây render, lại có `filters` mới, effect lại chạy. Cách 1 của em (clone) không sửa được vì clone cũng là địa chỉ mới. Cách 2 (bỏ khỏi dependency) chạy được nhưng đánh lừa React, và sẽ sai khi filters thay đổi thật. Các cách đúng:

```
// Cách 1: hằng số thì đưa ra ngoài component, chỉ tạo một lần
const FILTERS = { status: "open" };

// Cách 2: phụ thuộc vào primitive thay vì object
useEffect(() => {
  fetchTasks({ status }).then(setTasks);
}, [status]);
```

Ngày 2: Ôn value và reference

Bài ngày 2 khoảng 2 giờ. Làm đúng thứ tự, vẫn quy tắc viết dự đoán trước rồi mới chạy.

1. Viết lại 6 câu sai (30 phút). Với mỗi câu A1, A3, A4, A5, A6, A7, vẽ sơ đồ hộp và căn nhà ra giấy (biến nào trỏ vào đâu, sau mỗi dòng). Chụp ảnh nộp kèm.

2. Đoán output, bộ mới (30 phút). Mục tiêu 6/7.

```
// R1
const list = [{ id: 1, qty: 1 }];
const item = list.find((i) => i.id === 1);
item.qty = 5;
console.log(list[0].qty, item === list[0]);
```

```
// R2
let w = "hello";
w.toUpperCase();
console.log(w);

// R3
const a = [{ x: 1 }];
const b = [...a];
b.push({ x: 2 });
b[0].x = 9;
console.log(a.length, a[0].x);

// R4
function g(arr) { arr.push(1); arr = []; arr.push(2); }
const z = [];
g(z);
console.log(z);

// R5
const makeFilters = () => ({ status: "open" });
console.log(makeFilters() === makeFilters());

// R6
console.log(0 == "", 0 == "0", "" == "0");

// R7
const o = { a: 1 };
console.log(new Set([o, o]).size);
```

3. Sửa code (40 phút).

1. Sửa `shallowEqual` theo nhận xét, dùng `every` và `Object.hasOwn`. Viết 5 test bằng `console.log`, có trường hợp `{ a: undefined }` vs `{ b: 1 }`.
2. Thêm log chứng minh `updateCity` và `toggleTodo` không sửa input: in `user.address.city` cũ, và in `newTodos[1] === todos[1]` (phải là true với todo không đổi).
3. Viết hàm `increaseQty(items, id)` bằng `map` cho bài C1. Viết lại giải thích C1 và C2 trong 3 câu mỗi bài, bắt buộc dùng từ "địa chỉ".

4. Nhật ký và commit (20 phút). Lần trước em chưa nộp `log.md`, lần này nhớ thêm. Commit: `docs(js-core): day 02 review value vs reference`.